

```
import React, { useEffect, useRef, useCallback } from "react";
import { X, LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```
// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

```
// ⓘ LOCAL TYPE ISOLATION GATEWAY
export interface PopupActionConfig {
  label: string;
  onClick: () => void | Promise<void>;
  variant?: "brand" | "secondary" | "destructive";
  disabled?: boolean;
}
```

```
export interface PopupFormProps {
  isOpen: boolean;
  onClose: () => void;
  title: string;
  description?: string;
  icon?: LucideIcon;
  mode?: "modal" | "non-modal";
  width?: number | string;
  height?: number | string;
  headerActions?: Array<{
    icon: LucideIcon;
    onClick: () => void;
    tooltip?: string;
  }>;
  footerActions?: {
    primary?: PopupActionConfig;
    secondary?: PopupActionConfig;
  };
  children: React.ReactNode;
}
```

```
export function PopupForm({
  isOpen,
  onClose,
  title,
  description,
  icon: IconHeader,
  mode = "modal",
  width = 540,
  height,
  headerActions = [],
  footerActions,
```

```

    children,
  }: PopupFormProps) {
    const modalRef = useRef<HTMLDivElement>(null);
    const overlayRef = useRef<HTMLDivElement>(null);
    const previousActiveElementRef = useRef<HTMLElement | null>(null);
    const focusableElementsRef = useRef<HTMLElement[]>([]);

    const getFocusableElements = useCallback(() => {
      if (!modalRef.current) return [];
      const selector = [
        'button:not([disabled]):not([aria-hidden="true"])',
        '[href]:not([disabled]):not([aria-hidden="true"])',
        'input:not([disabled]):not([aria-hidden="true"])',
        'select:not([disabled]):not([aria-hidden="true"])',
        'textarea:not([disabled]):not([aria-hidden="true"])',
        '[tabindex]:not([tabindex="-1"]):not([disabled]):not([aria-hidden="true"])',
        '[contenteditable="true"]:not([disabled]):not([aria-hidden="true"])',
      ].join(",");
      return Array.from(modalRef.current.querySelectorAll<HTMLElement>(selector));
    }, []);

    const trapFocus = useCallback(
      (event: KeyboardEvent) => {
        if (event.key !== "Tab") return;
        const focusableElements = getFocusableElements();
        if (focusableElements.length === 0) return;

        const firstElement = focusableElements[0];
        const lastElement = focusableElements[focusableElements.length - 1];

        if (event.shiftKey) {
          if (document.activeElement === firstElement) {
            event.preventDefault();
            lastElement.focus();
          }
        } else {
          if (document.activeElement === lastElement) {
            event.preventDefault();
            firstElement.focus();
          }
        }
      },
      [getFocusableElements],
    );

    const handleKeyDown = useCallback(
      (event: KeyboardEvent) => {
        if (event.key === "Escape") {
          event.preventDefault();
          onClose();
        }
      },
    );
  }
}

```

```

        return;
    }
    trapFocus(event);
},
[onClose, trapFocus],
);

const handleOverlayClick = useCallback(
    (event: React.MouseEvent<HTMLDivElement>) => {
        if (mode === "modal" && event.target === overlayRef.current) {
            onClose();
        }
    },
    [mode, onClose],
);

useEffect(() => {
    if (isOpen) {
        previousActiveElementRef.current = document.activeElement as HTMLElement;
        document.addEventListener("keydown", handleKeyDown);
        document.body.style.overflow = "hidden";

        const focusableElements = getFocusableElements();
        focusableElementsRef.current = focusableElements;

        if (focusableElements.length > 0) {
            focusableElements[0].focus();
        } else {
            modalRef.current?.focus();
        }
    }

    return () => {
        document.removeEventListener("keydown", handleKeyDown);
        document.body.style.overflow = "";
        if (previousActiveElementRef.current) {
            previousActiveElementRef.current.focus();
        }
    };
}, [isOpen, handleKeyDown, getFocusableElements]);

useEffect(() => {
    if (isOpen && modalRef.current) {
        const focusableElements = getFocusableElements();
        focusableElementsRef.current = focusableElements;
        if (focusableElements.length > 0) {
            focusableElements[0].focus();
        }
    }
}, [isOpen, getFocusableElements]);

```

```

if (!isOpen) return null;

const containerWidth = typeof width === "number" ? `${width}px` : width;
const containerHeight = height
  ? typeof height === "number"
    ? `${height}px`
    : height
  : "auto";

const headerActionButtonButtons = headerActions.slice(0, 4).map((action, index) => (
  <button
    key={index}
    type="button"
    onClick={action.onClick}
    className={cn(
      "relative flex items-center justify-center",
      "w-8 h-8 rounded-lg",
      "text-muted-foreground/60 hover:text-foreground",
      "bg-transparent hover:bg-accent",
      "transition-all duration-150 ease-out",
      "active:scale-95",
      "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
      "disabled:opacity-50 disabled:pointer-events-none",
    )}
    aria-label={action.tooltip || "Action"}
  >
    <action.icon className="w-4 h-4" aria-hidden="true" />
  </button>
));

const closeButton = (
  <button
    type="button"
    onClick={onClose}
    className={cn(
      "relative flex items-center justify-center",
      "w-8 h-8 rounded-lg",
      "text-muted-foreground/60 hover:text-foreground",
      "bg-transparent hover:bg-accent",
      "transition-all duration-150 ease-out",
      "active:scale-95",
      "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
    )}
    aria-label="Close"
  >
    <X className="w-4 h-4" aria-hidden="true" />
  </button>

```

```

);

const renderFooterActions = () => {
  if (!footerActions) return null;

  const secondaryButton = footerActions.secondary ? (
    <button
      type="button"
      onClick={footerActions.secondary.onClick}
      disabled={footerActions.secondary.disabled}
      className={cn(
        "relative inline-flex items-center justify-center",
        "px-4 py-2.5 text-sm font-medium",
        "rounded-lg",
        "transition-all duration-150 ease-out",
        "active:scale-95",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring",
        "focus-visible:ring-offset-2",
        "disabled:opacity-50 disabled:pointer-events-none",
        footerActions.secondary.variant === "destructive"
          ? "text-destructive bg-destructive/10 hover:bg-destructive/20"
          : "text-muted-foreground bg-muted hover:bg-muted/80",
      )}
    >
      {footerActions.secondary.label}
    </button>
  ) : null;

  const primaryButton = footerActions.primary ? (
    <button
      type="button"
      onClick={footerActions.primary.onClick}
      disabled={footerActions.primary.disabled}
      className={cn(
        "relative inline-flex items-center justify-center",
        "px-4 py-2.5 text-sm font-medium",
        "rounded-lg",
        "transition-[var(--transition-fast)]",
        "active:scale-95",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring",
        "focus-visible:ring-offset-2",
        "disabled:opacity-50 disabled:pointer-events-none",
        "bg-brand-primary text-white",
        "hover:bg-brand-hover",
      )}
    >
      {footerActions.primary.label}
    </button>
  ) : null;

```

```

return (
  <div className="flex items-center justify-end gap-3">
    {secondaryButton}
    {primaryButton}
  </div>
);
};

return (
  <div
    ref={overlayRef}
    className={cn(
      "fixed inset-0 z-50 flex items-center justify-center",
      "p-4",
      mode === "modal"
        ? "bg-black/50 backdrop-blur-[var(--backdrop-blur)]"
        : "bg-transparent pointer-events-none",
      "transition-all duration-200 ease-out",
      "opacity-0 data-[state=open]:opacity-100",
    )}
    data-state={isOpen ? "open" : "closed"}
    onClick={handleOverlayClick}
    role={mode === "modal" ? "dialog" : "document"}
    aria-modal={mode === "modal" ? "true" : "false"}
    aria-labelledby="popup-form-title"
    aria-describedby={description ? "popup-form-description" : undefined}
  >
    <div
      ref={modalRef}
      tabIndex={-1}
      className={cn(
        "relative w-full max-h-[90vh] overflow-hidden",
        "bg-card backdrop-blur-[var(--backdrop-blur)]",
        "border border-[color-mix(in_oklch_var(--border)_50%_transparent)]",
        "rounded-surface-lg",
        "shadow-2xl",
        "font-sans",
        "flex flex-col",
        "transition-all duration-300 ease-out",
        "opacity-0 scale-95 data-[state=open]:opacity-100",
        "data-[state=open]:scale-100",
        "translate-y-4 data-[state=open]:translate-y-0",
      )}
      style={{
        width: containerWidth,
        height: containerHeight,
      }}
      data-state={isOpen ? "open" : "closed"}
      role={mode === "modal" ? "dialog" : "document"}
      aria-modal={mode === "modal" ? "true" : "false"}

```

```

>
<header
  className={cn(
    "flex items-start justify-between gap-4",
    "px-6 py-4",
    "border-b border-[color-mix(in_oklch_var(--border)_50%_transparent)]",
    "flex-shrink-0",
  )}
>
<div className="flex items-start gap-4 flex-1 min-w-0">
  {IconHeader && (
    <div
      className={cn(
        "flex-shrink-0 flex items-center justify-center",
        "w-10 h-10 rounded-lg",
        "bg-brand-primary/10",
        "text-brand-primary",
        "ring-1 ring-brand-primary/20",
      )}
      aria-hidden="true"
    >
      <IconHeader className="w-5 h-5" />
    </div>
  )}
  <div className="min-w-0">
    <h2
      id="popup-form-title"
      className={cn(
        "text-lg font-semibold text-card-foreground",
        "truncate",
        "font-sans",
      )}
    >
      {title}
    </h2>
    {description && (
      <p
        id="popup-form-description"
        className={cn(
          "mt-1 text-sm text-muted-foreground",
          "truncate",
          "font-sans",
        )}
      >
        {description}
      </p>
    )}
  </div>
</div>
<div className="flex items-center gap-2 flex-shrink-0">

```

```

        {headerActionButton}
        {closeButton}
      </div>
    </header>

    <main className={cn("flex-1 overflow-y-auto", "p-6", "font-sans")}>
      {children}
    </main>

    {footerActions && (
      <footer
        className={cn(
          "flex items-center justify-end gap-3",
          "px-6 py-4",
          "border-t border-[color-mix(in_oklch_var(--border)_50%_transparent)]",
          "bg-card/50 backdrop-blur-[var(--backdrop-blur)]",
          "flex-shrink-0",
        )}
      >
        {renderFooterActions()}
      </footer>
    )}
  </div>
</div>
);
}

export default PopupForm;

```